

Reinforcement Learning for Cybersecurity: A Flow-Based Defender for Binary Permit/Block Decisions

Javier Rivero Iglesias

May 2026

Contents

1	State of the Art	2
1.1	Network Intrusion Detection Systems	2
1.2	Flow-Based Traffic Representation	3
1.3	Public Datasets for Network Intrusion Detection	3
1.4	CICIDS2017 as the Main Internal Benchmark	4
1.5	Supervised Machine Learning and Deep Learning for NIDS	4
1.6	Reinforcement Learning Background	5
1.7	Deep Q-Learning and Distributional Reinforcement Learning	5
1.8	RL and DRL for Intrusion Detection	6
1.9	Classification-as-RL Formulation	6
1.10	Methodological Pitfalls in ML-Based NIDS	7
1.11	External Validation and Lab-Captured Traffic	7
1.12	Positioning of This Thesis	8

Chapter 1

State of the Art

This chapter reviews the research background for a reinforcement-learning-based defender that makes binary **PERMIT**/**BLOCK** decisions over network-flow observations. The objective is not to claim that reinforcement learning is new in intrusion detection. RL and DRL have already been studied for IDS and cyber-defense tasks, including DQN-style methods and broader autonomous-defense settings (Yang et al. 2024; Gueriani et al. 2024). The contribution of this thesis is instead scoped as a reproducible experimental prototype: a QRDQN-based flow-level defender with internal CICIDS2017 benchmarking, supervised baseline comparison, data-efficiency analysis, leakage-aware validation, and planned or preferred external validation using lab-captured traffic

1.1 Network Intrusion Detection Systems

Network Intrusion Detection Systems monitor network activity to identify traffic that may correspond to attacks, misuse, or policy violations (Scarfone and Mell 2007). A NIDS is part of a broader security-monitoring workflow: it observes packets, flows, or derived events; raises alerts or assigns labels; and supports later response. This distinction is important for this thesis because the current implementation studies offline detection and decision-making over recorded flow rows. It does not implement inline prevention, packet dropping, firewall updates, or operational response automation

Classical IDS taxonomies distinguish signature-based, anomaly-based, specification-based, and hybrid approaches (Scarfone and Mell 2007). Signature-based detection is effective when known malicious patterns can be matched, but it depends on updated rules and is less suited to novel behavior. Anomaly-based detection instead models expected behavior and flags deviations, which makes it attractive for machine learning but sensitive to distribution shift, noisy labels, and traffic evolution (Sperotto et al. 2010; Ring et al. 2019). Machine learning therefore provides useful tools for NIDS, but it also imports empirical ML risks: overfitting, leakage, weak validation, and poor generalization outside the training distribution (Arp et al. 2020; Layeghy and Portmann 2023).

The defender studied in this thesis uses access-decision vocabulary: **PERMIT** for benign flows and **BLOCK** for attack flows. This vocabulary connects model errors to operational meaning. A false negative would permit an attack flow, while a false positive would block benign traffic. In the present implementation, however, these decisions remain offline predictions over recorded or extracted flow rows, not active changes to network policy

1.2 Flow-Based Traffic Representation

Network traffic can be represented at several levels of granularity. Packet and payload inspection retain detailed protocol information, but they can be expensive, privacy-sensitive, and less practical when payloads are encrypted or unavailable. Flow-based representation aggregates packets into connection-like records and summarizes them through statistics such as duration, packet counts, byte counts, inter-arrival times, packet-size distributions, TCP flags, and activity or idle behavior (Sperotto et al. 2010; Habibi Lashkari et al. 2017).

Flow-based NIDS is attractive because it turns traffic into a tabular representation that can be processed at scale by statistical, machine-learning, and deep-learning methods (Sperotto et al. 2010; Liu and Lang 2019). It is also compatible with CICIDS2017, where PCAP traffic is converted into flow CSVs using CICFlowMeter-style feature extraction (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). In this thesis, the flow representation is the interface between captured traffic and the learning agent: each row is mapped into a canonical feature vector and presented as one observation

The same aggregation that makes flows practical also limits them. A flow record cannot express every payload-level or sequence-level signal that may matter for attack detection (Sperotto et al. 2010). Flow features can also encode dataset artifacts if preprocessing is careless. Identifiers, raw timestamps, IP addresses, and port fields can become label proxies rather than generalizable attack indicators (Arp et al. 2020). The repository therefore excludes IP addresses, absolute timestamps, Flow IDs, unique identifiers, and direct port-proxy fields from model features

The implemented input schema contains 76 canonical flow features and a 76-value missingness mask. The mask uses 1 for present or valid values and 0 for missing or imputed values, producing a 152-dimensional observation. This is a project-specific reproducibility mechanism, not a universal claim about optimal NIDS feature design.

1.3 Public Datasets for Network Intrusion Detection

Public NIDS datasets support shared benchmarking, reproducibility, and comparison across methods (Ring et al. 2019; Sharafaldin et al. 2018). They are also controlled artifacts. Their traffic reflects the generator, lab topology, feature extraction pipeline, labeling process, and time period used to create them. For that reason, benchmark performance should not be interpreted as direct evidence of deployment readiness (Ring et al. 2019; Apruzzese et al. 2022; Layeghy and Portmann 2023).

KDDCup99 and NSL-KDD are historically important because they shaped early IDS experimentation and standardized comparisons (KDD Cup 1999; Tavallaee et al. 2009). NSL-KDD attempted to address redundancy and evaluation problems in KDDCup99, but both datasets are now weak evidence for modern flow-based defense because their traffic, attack mix, and feature representation are outdated (Tavallaee et al. 2009; Ring et al. 2019). In this repository, NSL-KDD is preserved as historical benchmarking material, not as the final simulation-facing path

UNSW-NB15 is relevant as a more modern dataset generated in a cyber-range environment (Moustafa and Slay 2015). It helps contextualize the role of lab traffic and benchmark design, but it remains a controlled dataset rather than a guarantee of transfer to other networks. This distinction matters because a dataset can be more realistic than older benchmarks while still failing to represent the target distribution of a future

deployment (Apruzzese et al. 2022; Layeghy and Portmann 2023).

CICIDS2017 and CSE-CIC-IDS2018 are widely used CIC-family datasets for flow-based NIDS work (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017; Communications Security Establishment and Canadian Institute for Cybersecurity 2018). Bot-IoT and ToN_IoT provide additional IoT-oriented contexts and are useful secondary references when discussing dataset diversity beyond enterprise-style traffic (Koroniotis et al. 2019; Alsaedi et al. 2020). These datasets broaden the landscape, but they should not be used to imply that one public benchmark covers modern network behavior in general.

1.4 CICIDS2017 as the Main Internal Benchmark

CICIDS2017 is selected as the main internal benchmark because it provides labeled benign and attack traffic, PCAPs, and flow CSVs generated with CICFlowMeter-style features (Sharafaldin et al. 2018; Habibi Lashkari et al. 2017; Canadian Institute for Cybersecurity 2017). Compared with KDD-style datasets, it was designed to address recognized limitations in older IDS benchmarks and to provide more contemporary attack scenarios (Sharafaldin et al. 2018; Ring et al. 2019).

The dataset includes benign traffic and multiple attack families captured in a controlled environment (Sharafaldin et al. 2018; Canadian Institute for Cybersecurity 2017). The exact local class counts, row counts, and post-cleaning feature list still need to be verified against the curated CSVs before final reporting [**Verify: local CICIDS2017 curated CSV counts and schema**]. The repository keeps curated tracked CSVs and untracked raw exports for reference; the adapter performs cleaning, numeric coercion, infinite/NaN handling, and canonical mapping

CICIDS2017 is useful because it enables reproducible internal benchmarking. The repository already includes artifact-backed QRDQN training runs, Check A direct evaluation, Check B shuffled-label validation, and Check C hard CSV/day split validation. The hardest committed generalization artifact is Check C, and leave-one-exact-CSV-out validation exists in code but does not yet have a full committed artifact. The dataset should therefore be presented as an internal benchmark and validation ladder, not as proof of real-world readiness.

The limitations of CICIDS2017 are central to the thesis argument. It is lab-generated and class-imbalanced, and performance can be inflated by favorable random splits, duplicated or near-duplicated flows, leakage-prone fields, or preprocessing choices (Ring et al. 2019; Arp et al. 2020; Layeghy and Portmann 2023). A model that scores highly on a random CICIDS2017 split has not demonstrated production robustness. This thesis should separate internal benchmark evidence from stricter split validation and external lab-traffic evaluation (Apruzzese et al. 2022).

1.5 Supervised Machine Learning and Deep Learning for NIDS

Most NIDS work is naturally framed as supervised learning: a labeled traffic record is mapped to benign, attack, or a specific attack family (Liu and Lang 2019; Ring et al. 2019). Classical supervised methods such as Random Forest, decision trees, SVMs, and boosted trees remain important for tabular flow features, while deep-learning studies explore MLPs, CNNs, RNN/LSTM models, autoencoders, attention-based models, and

hybrids (Liu and Lang 2019). These models are not merely historical baselines. On structured flow features, simpler supervised methods can be strong and sometimes difficult for more complex models to beat under fair splits (Liu and Lang 2019; Apruzzese et al. 2022).

Random Forest is especially important here because it is a strong and natural baseline for tabular flow data. If an RL formulation is evaluated without strong supervised baselines, it becomes difficult to separate the value of the modeling framework from the value of the features and split protocol. The repository already includes a Random Forest baseline protocol aligned with the QRDQN evaluation splits, but metrics are still marked as pending

Deep learning NIDS studies broaden the design space, but they also illustrate common evaluation risks. Surveys report strong results across public datasets, yet many works depend on unclear splits, limited preprocessing detail, imbalanced labels, or no external validation (Liu and Lang 2019; Ring et al. 2019; Arp et al. 2020). This does not mean supervised ML/DL work should be dismissed. It means the comparison must be methodological rather than rhetorical: QRDQN should be evaluated under the same data, feature mapping, split, and metrics as supervised baselines before making any algorithmic claim

1.6 Reinforcement Learning Background

Reinforcement learning studies how an agent selects actions in an environment to maximize accumulated reward (Sutton and Barto 2018). The central elements are an observation or state, an action space, a reward signal, a policy, and a learning process that improves behavior from experience (Sutton and Barto 2018). In classical control problems, actions influence future states, making the problem explicitly sequential.

This thesis uses RL in a narrower experimental setting. Each flow row is an observation; the action is binary; and the reward is computed from the chosen action and the ground-truth label. The actions are 0 = PERMIT / BENIGN and 1 = BLOCK / ATTACK. This makes cost-sensitive decision design explicit: false negatives and false positives can be penalized differently (Lee et al. 2002)

The limitation is that a static labeled dataset does not automatically provide rich temporal dynamics. Without additional state evolution or adversarial interaction, the RL problem may resemble reward-engineered classification (Sutton and Barto 2018; Farama Foundation 2026). This is acceptable only if the thesis states the trade-off clearly and keeps supervised baseline comparison central

1.7 Deep Q-Learning and Distributional Reinforcement Learning

Deep Q-Learning uses neural networks to approximate action-value functions (Mnih et al. 2015). DQN popularized experience replay and target networks, two stabilization mechanisms that remain central in value-based deep RL (Mnih et al. 2015). Double DQN addresses overestimation bias by decoupling action selection from target evaluation (Hasselt et al. 2016), and dueling architectures separate value and advantage estimation (Wang et al. 2016).

Distributional reinforcement learning estimates a distribution over possible returns rather than only the expected return (Bellemare et al. 2017). This perspective is con-

ceptually relevant to security decisions because false positives and false negatives do not carry the same meaning. However, modeling a return distribution does not by itself prove risk-sensitive behavior or better NIDS performance (Bellemare et al. 2017; Dabney et al. 2018).

QRDQN approximates the return distribution with quantile regression (Dabney et al. 2018). In this thesis it is a defensible experimental choice for a binary flow-level defender with asymmetric rewards (Stable-Baselines3 Contributors 2026; Farama Foundation 2026). It should not be described as proven superior for NIDS unless same-protocol comparisons against DQN, Random Forest, and other baselines support that claim

1.8 RL and DRL for Intrusion Detection

RL and DRL have already been applied to intrusion detection and cyber defense (Yang et al. 2024; Gueriani et al. 2024). Existing work includes dataset-as-environment formulations, DQN-style classifiers, actor-critic methods, adversarial-environment designs, and broader autonomous-defense settings (Yang et al. 2024; Gueriani et al. 2024). These survey sources are useful for positioning, but they are preprints in the current bibliography; final thesis claims about specific prior systems should audit original papers individually.

The processed research base suggests that many RL-for-IDS works remain heterogeneous in dataset choice, split protocol, baseline comparison, and external validation. This claim should be treated as a literature-synthesis claim, not as a numerical result, until paper-level audit is completed [**Verify: original RL-IDS papers and protocols**]. The thesis should therefore avoid novelty overclaims. The relevant claim is not that RL for IDS is absent, but that this project studies a specific flow-level binary PERMIT/BLOCK formulation with QRDQN under a reproducible evaluation protocol

Autonomous cyber-defense work is related but distinct. Many autonomous-defense studies model defenders that select response strategies in simulated networks or attack graphs [**Citation needed: audited autonomous cyber-defense survey**]. The present thesis is closer to an IDS decision problem exposed through a Gymnasium-compatible environment. It can be framed as a building block for future defense automation, not as a complete autonomous defense system (Farama Foundation 2026)

1.9 Classification-as-RL Formulation

The main methodological tension is that labeled flow detection is naturally a supervised classification problem, while this project formulates it as an RL environment. The favorable argument is that binary security decisions have asymmetric costs: false negatives permit attacks, while false positives block benign traffic. RL rewards can encode these costs directly (Lee et al. 2002)

A second favorable argument is continuity with future sequential defense. A PERMIT/BLOCK interface resembles a decision that an eventual defender might make, even though the current implementation is offline. A Gymnasium-compatible environment also provides a structured interface that could later be extended with temporal context, online feedback, richer actions, or simulated adversarial behavior (Farama Foundation 2026).

The critical argument is that if each observation is an independent labeled row and reward mirrors the label, RL may add complexity without clear modeling benefit over supervised learning (Sutton and Barto 2018; Liu and Lang 2019). In that case, supervised

baselines are mandatory. The correct framing is balanced: the thesis studies classification-as-RL because it makes costs and the PERMIT/BLOCK interface explicit, but the experiments must show how this formulation compares with supervised baselines

1.10 Methodological Pitfalls in ML-Based NIDS

NIDS evaluation is vulnerable to inflated metrics. Random row-wise splits can place related flows, repeated patterns, or scenario-specific artifacts in both training and test data, making the test set less independent than it appears (Arp et al. 2020; Apruzzese et al. 2022; Layeghy and Portmann 2023). Very high accuracy is therefore not sufficient evidence unless the split, preprocessing, class distribution, and leakage controls are clear (Arp et al. 2020; Ring et al. 2019).

Data leakage is a central risk. Identifiers, IP addresses, absolute timestamps, Flow IDs, or ports that act as label proxies can cause a model to learn dataset artifacts rather than attack behavior (Arp et al. 2020). The repository addresses this through an anti-leakage policy and loader-level cleaning. It also includes a shuffled-label validation artifact where performance did not remain artificially high under label permutation. This reduces concern about obvious leakage in that run, but it is not exhaustive proof against all leakage (Arp et al. 2020).

Class imbalance is another concern. Accuracy can hide poor minority-class detection or unacceptable false-positive behavior (Ring et al. 2019; Liu and Lang 2019). This thesis should therefore emphasize per-class precision, recall, F1, TP, FP, FN, TN, false-positive rate, and false-negative rate. Attack-family error analysis should be added if final artifacts preserve family labels [**Citation needed: attack-family evaluation source**].

Reproducibility is also a methodological requirement. A thesis result should be tied to a run configuration, split definition, scaler, preprocessing policy, random seed, and persisted metrics. This repository already frames meaningful runs through config and metrics artifacts, but some gaps remain: Random Forest metrics are pending, the full leave-one-exact-CSV-out artifact is not committed, and reward documentation has a mismatch that should be resolved before final methodology prose.

1.11 External Validation and Lab-Captured Traffic

External validation matters because public-dataset performance may not transfer to other traffic distributions (Apruzzese et al. 2022; Layeghy and Portmann 2023). A model trained on CICIDS2017 may learn patterns specific to that capture environment, attack scripts, feature extraction pipeline, or class distribution (Ring et al. 2019; Arp et al. 2020). Testing on private lab-captured traffic can provide an additional distribution-shift check if the same flow-extraction and canonical-mapping pipeline is used.

The current Phase 2 pipeline is offline inference. It extracts or receives flow CSVs, maps them to the canonical schema, loads the trained model and scaler, applies robust preprocessing options, and produces predictions and diagnostics. It does not implement live network enforcement, packet dropping, firewall rule updates, or active inline blocking.

Existing Phase 2 artifacts must be interpreted cautiously. The results snapshot records two benign-only runs with different behavior: an early run blocked all benign flows, while a later run allowed all benign flows under different conditions. This is why Phase 2 claims must cite exact run IDs and configurations. A low-volume or benign-only lab capture can

be a false-positive and domain-shift sanity check, but it is not equivalent to full external validation across benign and attack traffic (Layeghy and Portmann 2023).

1.12 Positioning of This Thesis

This thesis is best positioned as a carefully scoped experimental prototype. RL and DRL for intrusion detection already exist, public NIDS benchmarks are common, and supervised ML/DL methods remain strong baselines (Yang et al. 2024; Gueriani et al. 2024; Liu and Lang 2019). The contribution is not “the first RL IDS” and not proof that QRDQN is the best algorithm for NIDS.

The contribution is a reproducible formulation and evaluation pipeline for a binary flow-level defender. The project maps CICIDS2017 and lab-flow inputs into a fixed 76-feature canonical schema, augments observations with a missingness mask to produce 152-dimensional inputs, exposes each row through a Gymnasium-style dataset-as-environment, and trains a QRDQN agent to choose between **PERMIT** and **BLOCK** (Farama Foundation 2026; Stable-Baselines3 Contributors 2026). The design makes false-positive and false-negative consequences explicit through reward shaping, while preserving the need for supervised baseline comparison (Lee et al. 2002).

The evaluation posture should be skeptical by design. Random-split performance is useful but not sufficient; strict splits, shuffled-label validation, leave-one-exact-CSV-out evaluation, and external lab-flow inference each test different failure modes (Apruzzese et al. 2022; Layeghy and Portmann 2023). In the current repository, Random Forest metrics and the full leave-one-exact-CSV-out artifact remain gaps, Phase 2 behavior must be tied to exact run artifacts, and external validation should be described as planned or preferred unless a current artifact supports a stronger claim

Bibliography

- Alsaedi, Abdullah, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar (2020). “TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems”. In: *IEEE Access* 8, pp. 165130–165150. DOI: 10.1109/ACCESS.2020.3022862.
- Apruzzese, Giovanni, Luca Pajola, and Mauro Conti (2022). “The Cross-Evaluation of Machine Learning-Based Network Intrusion Detection Systems”. In: *IEEE Transactions on Network and Service Management* 19.4, pp. 5152–5169. DOI: 10.1109/TNSM.2022.3157344.
- Arp, Daniel, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck (2020). “Dos and Don’ts of Machine Learning in Computer Security”. In: arXiv: 2010.09470 [cs.CR].
- Bellemare, Marc G., Will Dabney, and Rémi Munos (2017). “A Distributional Perspective on Reinforcement Learning”. In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70, pp. 449–458. URL: <https://proceedings.mlr.press/v70/bellemare17a.html>.
- Canadian Institute for Cybersecurity (2017). *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. Dataset documentation page. URL: <https://www.unb.ca/cic/datasets/ids-2017.html>.
- Communications Security Establishment and Canadian Institute for Cybersecurity (2018). *A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)*. URL: <https://registry.opendata.aws/cse-cic-ids2018/>.
- Dabney, Will, Mark Rowland, Marc G. Bellemare, and Rémi Munos (2018). “Distributional Reinforcement Learning with Quantile Regression”. In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/11791>.
- Farama Foundation (2026). *Gymnasium Documentation*. Software documentation. URL: <https://gymnasium.farama.org/>.
- Gueriani, Afrah, Hamza Kheddar, and Ahmed Cherif Mazari (2024). “Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey”. In: Preprint; use cautiously until publication status is audited. arXiv: 2405.20038 [cs.CR].
- Habibi Lashkari, Arash, Gerard Draper Gil, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani (2017). “Characterization of Tor Traffic Using Time Based Features”. In: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy*, pp. 253–262. DOI: 10.5220/0006105602530262.
- Hasselt, Hado van, Arthur Guez, and David Silver (2016). “Deep Reinforcement Learning with Double Q-Learning”. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100. arXiv: 1509.06461.

- KDD Cup (1999). *KDD Cup 1999 Data*. Dataset page; use only for historical dataset identification. URL: <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>.
- Koroniotis, Nickolaos, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull (2019). “Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset”. In: *Future Generation Computer Systems* 100, pp. 779–796. DOI: 10.1016/j.future.2019.05.041.
- Layeghy, Siamak and Marius Portmann (2023). “Explainable Cross-Domain Evaluation of ML-Based Network Intrusion Detection Systems”. In: *Computers and Electrical Engineering* 108, p. 108692. DOI: 10.1016/j.compeleceng.2023.108692.
- Lee, Wenke, Wei Fan, Matthew Miller, Salvatore J. Stolfo, and Erez Zadok (2002). “Toward Cost-Sensitive Modeling for Intrusion Detection and Response”. In: *Journal of Computer Security* 10.1-2. Use for cost-sensitive IDS concept; final metadata should be checked against library database, pp. 5–22.
- Liu, Hongyu and Bo Lang (2019). “Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey”. In: *Applied Sciences* 9.20, p. 4396. DOI: 10.3390/app9204396.
- Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis (2015). “Human-Level Control through Deep Reinforcement Learning”. In: *Nature* 518.7540, pp. 529–533. DOI: 10.1038/nature14236.
- Moustafa, Nour and Jill Slay (2015). “UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)”. In: *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942.
- Ring, Markus, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho (2019). “A Survey of Network-Based Intrusion Detection Data Sets”. In: *Computers & Security* 86, pp. 147–167. DOI: 10.1016/j.cose.2019.06.005. arXiv: 1903.02460.
- Scarfone, Karen A. and Peter M. Mell (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)*. Tech. rep. Special Publication 800-94. National Institute of Standards and Technology. URL: <https://csrc.nist.gov/pubs/sp/800/94/final>.
- Sharafaldin, Iman, Arash Habibi Lashkari, and Ali A. Ghorbani (2018). “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization”. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pp. 108–116. DOI: 10.5220/0006639801080116. URL: <https://www.unb.ca/cic/datasets/ids-2017.html>.
- Sperotto, Anna, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras, and Burkhard Stiller (2010). “An Overview of IP Flow-Based Intrusion Detection”. In: *IEEE Communications Surveys & Tutorials* 12.3, pp. 343–356. DOI: 10.1109/SURV.2010.032210.00054.
- Stable-Baselines3 Contributors (2026). *QR-DQN: Stable Baselines3 Contrib Documentation*. Software documentation. URL: <https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html>.
- Sutton, Richard S. and Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.

- Tavallaee, Mahbod, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani (2009). “A Detailed Analysis of the KDD CUP 99 Data Set”. In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.
- Wang, Ziyu, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas (2016). “Dueling Network Architectures for Deep Reinforcement Learning”. In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48, pp. 1995–2003. URL: <https://proceedings.mlr.press/v48/wangf16.html>.
- Yang, Wanrong, Alberto Acuto, Yihang Zhou, and Dominik Wojtczak (2024). “A Survey for Deep Reinforcement Learning Based Network Intrusion Detection”. In: Preprint; use cautiously until publication status is audited. arXiv: 2410.07612 [cs.CR].